International Institute of Information Technology, Bhubaneswar

(A University established by Government of Odisha)

Adyasha Panda *
gmail:adyashapanda

Anirudha Bose *
gmail:ani07nov

* Department of Computer Science & Engineering
IIIT, Bhubaneswar

Status of this memo

This memo is a report on the project **PageRank Computation in Structured P2P Networks - an implementation using MapReduce technique in Hadoop**.

Last updated on 18th November, 2014

Mentor

Prof. Bharati Mishra
Assistant Professor, Department of Computer Science & Engineering
IIIT, Bhubaneswar

Abstract

In a Peer-To-Peer (P2P) network system, autonomous computers pool their resources (e.g., files, storage, compute cycles) to inexpensively handle tasks that would normally require costly servers. It's performance is affected due to the scale of these systems, their "open nature," and the lack of centralized control. Resource allocation and query routing is difficult in such networks. Various algorithms have been implemented by researchers to optimize the resource allocation and query routing [1]. Ranking the peers is an effective way to resolve this issue [2,3,4].

This project focuses on the ranking problem of peer-to-peer networks. Google's PageRank algorithm used to rank websites in their search engine is highly efficient. In this project the PageRank algorithm is implemented using MapReduce in a single node Hadoop cluster, in a peer-to-peer network, to allocate rank to the peers in the network.

PageRank Algorithm

Google has introduced a pagerank algorithm to rank the websites in its search engine .PageRank is a link algorithm and it assigns a numerical weighting to each element of a hyperlinked set of documents, such as the World Wide Web, with the purpose of "measuring" its relative importance within the set. Fundamentally, PageRank gives pages that have a large number of inbound links a higher score than pages that have a smaller number of inbound links. When evaluating the score for a page, PageRank uses the scores for all the inbound links to calculate a page's PageRank. But it penalizes individual inbound links from sources that have a high number of outbound links by dividing that outbound link PageRank by the number of outbound links.

The original PageRank algorithm was described by Lawrence Page and Sergey Brin in several publications.

$$PR(A) = (1-d) + d\ (PR(T1)/C(T1) + \ldots + PR(Tn)/C(Tn))$$

where

PR(A) is the PageRank of page A,
PR(Ti) is the PageRank of pages Ti which link to page A,
C(Ti) is the number of outbound links on page Ti and
d is a damping factor which can be set between 0 and 1. [12]

Map and Reduce

MapReduce is a programming model of a big data processing tool, Hadoop, used to process large datasets in a parallel distributed programming approach. It consists of 2 phases: Map and Reduce. In the mapping phase the whole dataset is filtered and sorted, and in the Reduce phase the result is summed up. In the Map and Reduce functions the data is structured in (key,value) pairs. The Map function accepts input in the form of a value associated with each key and returns a list of values associated with every key.

```
Map(k1,v1) → list(k2,v2)
```

The Map function is applied in parallel to the various segments of the dataset and the result from each Map function is then collected ,sorted ,shuffled and fed into the Reduce function .
The reduce function accepts the output from the Map function ,sums up the list of values associated with a key according to the problem statement and returns a value associated with each key.

```
Reduce [list(k2,v2)] → (k3,v3)
```

PageRank Algorithm in MapReduce

```
Map(node-name, node)
emit(node-name, node) { preserve the graph structure }
outPageRank ← node.pageRank / |node.adjecency-list|
for all adj E node.adjecency-list do
      emit(adjnode.name,outPageRank)


Reduce(node-name, [node, inPageRank1, inPageRank2, … ])
sumInPageRanks ← 0
```

```
node ← null
for all i E [node, inPageRank1, inPageRank2, … } do
      if i isa node then
            node  i
      else
            sumInPageRank ← sumInPageRanks + i
m.pageRank ← sumInPageRanks
emit(node-name, node)
```

In the Map Function each node is a key and has an associated list of values associated with it. Initially all the nodes in the graph are initialised with a default value which is equal to 1 divided by total number of nodes in the graph.

With each iteration, every node adds some pagerank value to all its adjacent nodes through the outbound link which is equal to the PageRank of the node divided by the number of adjacent nodes.

Then the new PageRank values are calculated according to the formula:

$$\text{PageRank}(n) = \frac{1-d}{|webGraph|} + d \sum_{inboundLinks\ (n)} \frac{PageRank(i)}{|i.outboundLinks|}$$

The Map phase is responsible for preserving the graph structure as well as emitting the PageRank value to all the outbound nodes. The Map outputs two pieces of information : the original graph and the outbound PageRank values. The Reduce phase is responsible for recalculating the new PageRank value for each node and including it in the output of the original graph. Its job is to reconstruct the original graph and to update each node with a new PageRank value based on all the sum of inbound PageRank values.

The final piece is a driver class that will keep launching the MapReduce job and read the counter from each job until you have convergence. Convergence value is set for small graphs where the graph is considered to have converged if the pageRank of all nodes changes by a certain convergence value from its initial value. In case of large graphs a fixed number of iterations is fixed for the graph, at the end of which the graph is considered to have converged. In this project, a single node hadoop cluster is used for processing the dataset. The number of nodes for which the PageRank is being calculated is large. Considering this, here the number of iterations is fixed to achieve convergence.

The probability distribution in PageRank algorithm

In the PageRank formula, d set to 0.85, is a constant damping factor used in two parts. First, it denotes the probability of a random surfer reaching the page after clicking on many links and second, it dampens the effect of the inbound link PageRanks by 85%. Many research papers have been published to analyze the effect of probability distribution or the damping factor on the PageRank [6,7,8]. According to [9] "Generalisation of the Damping Factor in PageRank for Weighted Networks", the damping factor has been generalised for weighted networks. The damping factor used in Google's PageRank algorithm was calculated keeping in view the entire web world consisting of approximately 10^9 webpages. But in our dataset we have used only 10000 nodes approximately. Thus according to our dataset, the probability distribution or the damping factor is:

```
D = 0.85^[log 10^9 / log 10^4] = 0.69
```

Dataset

The dataset used to implement MapReduce is a peer-to-peer network with 10,876 nodes and 39994 edges. The dataset is taken from the Stanford Network Analysis Project [5]. Nodes represent hosts in the Gnutella network topology and edges represent connections between the Gnutella hosts. Snapshots of the Gnutella peer-to-peer file sharing network were taken in August 2002.

Dataset Statistics:

Nodes	10876

Edges	39994
Nodes in largest WCC	10876 (1.000)
Edges in largest WCC	39994 (1.000)
Nodes in largest SCC	4317 (0.397)
Edges in largest SCC	18742 (0.469)
Average clustering coefficient	0.0062
Number of triangles	934
Fraction of closed triangles	0.001807
Diameter (longest shortest path)	9
90-percentile effective	5.4

The dataset specifies the edges connecting the nodes. It is a directed graph and in the following format:

```
# FromNodeId   ToNodeId
0              1
0              2
0              3
0              4
1              17
1              18
1              19
3              20
3              21
3              22
3              23
```

In order to implement MapReduce, this data needs to be cleaned and arranged in a specific manner. Each node can be used as a key in MapReduce with a list of values associated with it denoting the outbound links. At the time of taking a snapshot of the dataset, the nodes [10452, 10493, 10647] were dead and didn't accept incoming connections from other peers. In their stead, three external nodes are present in the dataset which are numbered as [10876, 10877, 10878]. The total number of nodes are still 10876 as specified in the dataset statistics. The data is in the following format:

```
# Node  OutboundLinks
0       1       2       3       4
1       17      18      19
3       20      21      22      23
```

Cleaning of the data:

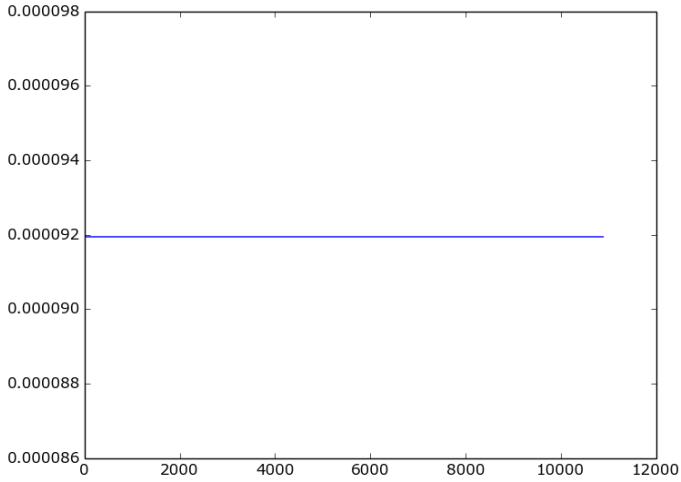
We used a simple Python script to break the dataset in the above form.

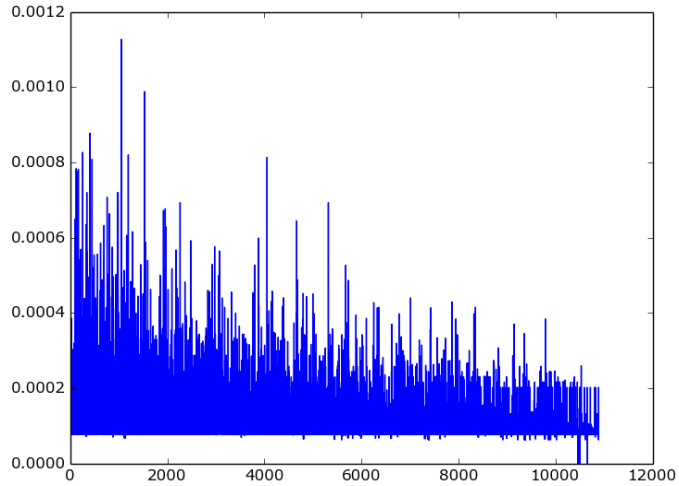
```
data = open("/home/ani/p2p-Gnutella04.txt", "r").read()
d = dict()
for each in data.split("\n"):
    if each.split()[0] in d:
        l.append(each.split()[1])
    else:
        l = []
        l.append(each.split()[1])
    d[each.split()[0]] = l
```

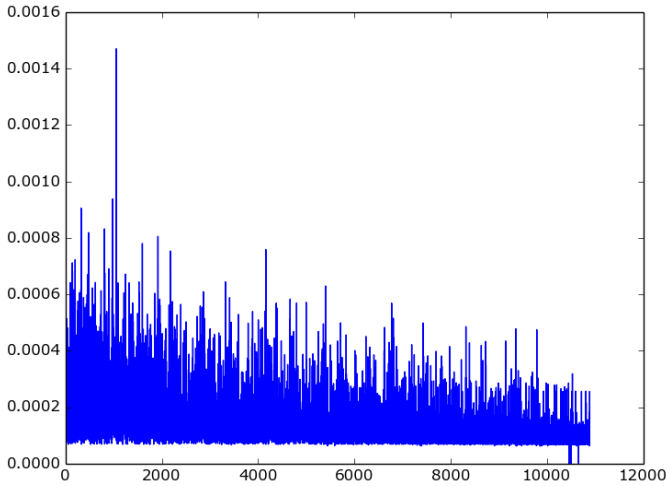
```
for each in d:
    print each + " " + " ".join(d[each])
```

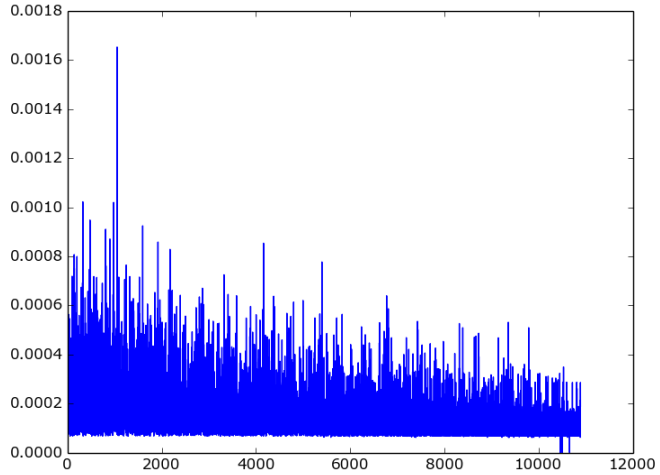
Graphs

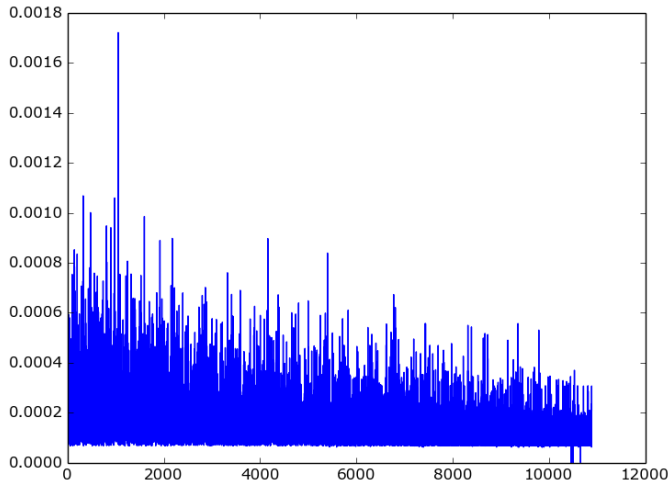
The nodes representing the hosts in the Gnutella network topology are plotted with respect to their PageRanks after every iteration. In our implementation, we have considered a total of 5 iterations.

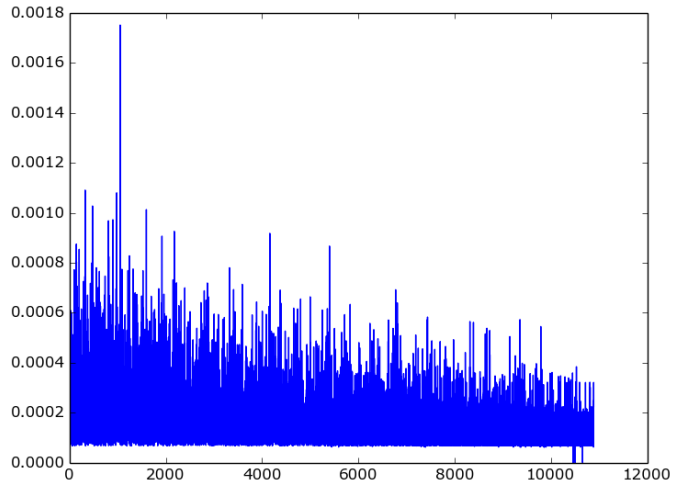












Conclusion

The Google's PageRank algorithm in MapReduce model efficiently ranked the nodes of the network in the dataset, representing the hosts in the Gnutella P2P network topology. We considered 5 iterations of the algorithm and plotted the graphs for each iteration. We can conclude from the graph that the change in PageRank value in every iteration is greater for the nodes with higher number of inbound links than those nodes with lower number of inbound links. The PageRank value is zero for dead nodes, with no inbound and outbound links to and from other peer nodes. The function in which the PageRank values change in each iteration is a study of interest and we are currently working on it.

Google's PageRank algorithm in MapReduce model can be implemented in various similar structured graphs to rank the nodes, for example, to rank the airports in a network where the nodes represent the airports and the in-flight and out-flight aeroplanes represent the inbound and outbound links, to rank the nodes in collaboration networks where the scientists represent the nodes and the edges represent collaborations (co-authoring a paper), to rank the nodes of road network where the nodes represent cities and the edges represent the roads connecting them. Various best practises for Mapreduce graph algorithms with respect to partitioning, serializing, and distributing the graph and message passing could be implemented to optimize the run-time and throughput.

Appendix

Implementation in Java:

File: **main.java**

```
package com.pagerank;

import org.apache.commons.io.*;
import org.apache.commons.lang.StringUtils;
import org.apache.hadoop.conf.Configuration;
import org.apache.hadoop.fs.FileSystem;
import org.apache.hadoop.fs.*;
import org.apache.hadoop.io.Text;
import org.apache.hadoop.mapreduce.Job;
import org.apache.hadoop.mapreduce.lib.input.*;
import org.apache.hadoop.mapreduce.lib.output.FileOutputFormat;

import java.io.*;
import java.util.*;

public final class Main
```

```
{
  public static void main(String... args) throws Exception
  {

    String inputFile = args[0];
    String outputDir = args[1];

    iterate(inputFile, outputDir);
  }

  public static void iterate(String input, String output) throws Exception
  {
    Configuration conf = new Configuration();
    Path outputPath = new Path(output);
    outputPath.getFileSystem(conf).delete(outputPath, true);
    outputPath.getFileSystem(conf).mkdirs(outputPath);

    Path inputPath = new Path(outputPath, "input.txt");

    int numNodes = createInputFile(new Path(input), inputPath);

    int iter = 1;
    double desiredConvergence = 0.00001;

    while (true)
    {
      Path jobOutputPath = new Path(outputPath, String.valueOf(iter));

      System.out.println("+-----------------------------------+");
      System.out.println("|  Iteration:    " + iter);
      System.out.println("|  Input path:   " + inputPath);
      System.out.println("|  Output path:  " + jobOutputPath);
      System.out.println("+-----------------------------------+");

      if (calcPageRank(inputPath, jobOutputPath, numNodes) <
          desiredConvergence) {
        System.out.println(
            "Convergence is below " + desiredConvergence +
                ", we're done");
        break;
      }
      inputPath = jobOutputPath;
      iter++;

       if(iter > 5)
       { break; }
    }
  }

  public static int createInputFile(Path file, Path targetFile) throws IOException
  {
    Configuration conf = new Configuration();
    FileSystem fs = file.getFileSystem(conf);

    int numNodes = getNumNodes(file);
    double initialPageRank = 1.0 / (double) numNodes;

    OutputStream os = fs.create(targetFile);
    LineIterator iter = IOUtils.lineIterator(fs.open(file), "UTF8");
```

```java
  while (iter.hasNext())
  {
    String line = iter.nextLine();

    String[] parts = StringUtils.split(line);

    Node node = new Node().setPageRank(initialPageRank).setAdjacentNodeNames(
            Arrays.copyOfRange(parts, 1, parts.length));
    IOUtils.write(parts[0] + '\t' + node.toString() + '\n', os);
  }
  os.close();
  return numNodes;
}

public static int getNumNodes(Path file) throws IOException
{
  Configuration conf = new Configuration();
  FileSystem fs = file.getFileSystem(conf);

  return IOUtils.readLines(fs.open(file), "UTF8").size();
}

public static double calcPageRank(Path inputPath, Path outputPath, int numNodes)
    throws Exception
{
  Configuration conf = new Configuration();
  conf.setInt(Reduce.CONF_NUM_NODES_GRAPH, numNodes);

  Job job = new Job(conf);
  job.setJarByClass(Main.class);
  job.setMapperClass(Map.class);
  job.setReducerClass(Reduce.class);

  job.setInputFormatClass(KeyValueTextInputFormat.class);

  job.setMapOutputKeyClass(Text.class);
  job.setMapOutputValueClass(Text.class);

  FileInputFormat.setInputPaths(job, inputPath);
  FileOutputFormat.setOutputPath(job, outputPath);

  if (!job.waitForCompletion(true))
  {
    throw new Exception("Job failed");
  }

  long summedConvergence = job.getCounters().findCounter(
      Reduce.Counter.CONV_DELTAS).getValue();
  double convergence =
      ((double) summedConvergence /
          Reduce.CONVERGENCE_SCALING_FACTOR) /
          (double) numNodes;

  System.out.println("+-----------------------------------+");
  System.out.println("|  Num nodes:           " + numNodes);
  System.out.println("|  Summed convergence:  " + summedConvergence);
  System.out.println("|  Convergence:         " + convergence);
  System.out.println("+-----------------------------------+");
```

```
    return convergence;
  }
}
```

File: **map.java**

```
package com.pagerank;

import org.apache.hadoop.io.Text;
import org.apache.hadoop.mapreduce.Mapper;

import java.io.IOException;

public class Map extends Mapper<Text, Text, Text, Text>
{
  private Text outKey = new Text();
  private Text outValue = new Text();

  @Override
  protected void map(Text key, Text value, Context context)
      throws IOException, InterruptedException
  {
    context.write(key, value);
    Node node = Node.fromMR(value.toString());

    if(node.getAdjacentNodeNames() != null && node.getAdjacentNodeNames().length > 0)
    {
      double outboundPageRank = node.getPageRank() /
                                (double)node.getAdjacentNodeNames().length;

      // go through all the nodes and propagate PageRank to them
      for (int i = 0; i < node.getAdjacentNodeNames().length; i++)
      {
        String neighbor = node.getAdjacentNodeNames()[i];
        outKey.set(neighbor);
        Node adjacentNode = new Node().setPageRank(outboundPageRank);

        outValue.set(adjacentNode.toString());
        System.out.println("output -> K[" + outKey + "],V[" + outValue + "]");
        context.write(outKey, outValue);
      }
    }
  }
}
```

File: **reduce.java**

```
package com.pagerank;

import org.apache.hadoop.io.Text;
import org.apache.hadoop.mapreduce.Reducer;

import java.io.IOException;

public class Reduce extends Reducer<Text, Text, Text, Text>
{
  public static final double CONVERGENCE_SCALING_FACTOR = 1000.0;
```

```java
  public static final double DAMPING_FACTOR = 0.69;
  public static String CONF_NUM_NODES_GRAPH = "pagerank.numnodes";
  private int numberOfNodesInGraph;

  public static enum Counter
  {
    CONV_DELTAS
  }

  @Override
  protected void setup(Context context) throws IOException, InterruptedException
  {
    numberOfNodesInGraph = context.getConfiguration().getInt(CONF_NUM_NODES_GRAPH, 0);
  }
  private Text outValue = new Text();
  public void reduce(Text key, Iterable<Text> values, Context context)
      throws IOException, InterruptedException
  {
    System.out.println("input -> K[" + key + "]");
    double summedPageRanks = 0;
    Node originalNode = new Node();
    for (Text textValue : values)
    {
      System.out.println("input -> V[" + textValue + "]");
      Node node = Node.fromMR(textValue.toString());
      if (node.containsAdjacentNodes())
      {
        originalNode = node;
      }
      else
      {
        summedPageRanks += node.getPageRank();
      }
    }
    double dampingFactor = ((1.0 - DAMPING_FACTOR) / (double) numberOfNodesInGraph);

    double newPageRank = dampingFactor + (DAMPING_FACTOR * summedPageRanks);

    double delta = originalNode.getPageRank() - newPageRank;

    originalNode.setPageRank(newPageRank);

    outValue.set(originalNode.toString());

    System.out.println("output -> K[" + key + "],V[" + outValue + "]");
    context.write(key, outValue);
    int scaledDelta = Math.abs((int) (delta * CONVERGENCE_SCALING_FACTOR));

    System.out.println("Delta = " + scaledDelta);
    context.getCounter(Counter.CONV_DELTAS).increment(scaledDelta);
  }
}
```

File: **node.java**

```java
package com.pagerank;

import org.apache.commons.lang.StringUtils;
```

```java
import java.io.IOException;
import java.util.Arrays;

public class Node
{
  private double pageRank = 0.25;
  private String[] adjacentNodeNames;

  public static final char fieldSeparator = '\t';

  public double getPageRank()
  {
    return pageRank;
  }

  public Node setPageRank(double pageRank)
  {
    this.pageRank = pageRank;
    return this;
  }

  public String[] getAdjacentNodeNames()
  {
    return adjacentNodeNames;
  }

  public Node setAdjacentNodeNames(String[] adjacentNodeNames)
  {
    this.adjacentNodeNames = adjacentNodeNames;
    return this;
  }

  public boolean containsAdjacentNodes()
  {
    return adjacentNodeNames != null;
  }

  @Override
  public String toString()
  {
    StringBuilder sb = new StringBuilder();
    sb.append(pageRank);

    if (getAdjacentNodeNames() != null)
    {
      sb.append(fieldSeparator)
          .append(StringUtils
              .join(getAdjacentNodeNames(), fieldSeparator));
    }
    return sb.toString();
  }

  public static Node fromMR(String value) throws IOException
  {
    String[] parts = StringUtils.splitPreserveAllTokens(value, fieldSeparator);
    if (parts.length < 1)
    {
      throw new IOException("Expected 1 or more parts but received " + parts.length);
    }
```

```
    Node node = new Node().setPageRank(Double.valueOf(parts[0]));
    if (parts.length > 1)
    {
      node.setAdjacentNodeNames(Arrays.copyOfRange(parts, 1, parts.length));
    }
    return node;
  }
```

Setting up a single node hadoop cluster:

In the project a pseudo distributed single node hadoop cluster backed up with hadoop distributed file system has been installed and run in ubuntu linux. Hadoop is a framework written in Java for running applications on large clusters of commodity hardware and incorporates features similar to those of the Google File System (GFS) and of the MapReduce computing paradigm. Hadoop's HDFS is a highly fault-tolerant distributed file system and, like Hadoop in general, designed to be deployed on low-cost hardware. It provides high throughput access to application data and is suitable for applications that have large data sets. Java is needed to run hadoop on the machine. To install hadoop and run it, a dedicated hadoop system user is added, then SSH is configured and connected to the local machine. Hadoop is downloaded from Apache Download mirrors and HOME/.bashrc is updated. The environment variables are configured according to hadoop. Now hadoop installation is ready to be started. The HDFS is formatted the first time hadoop is used. [10]

Running the MapReduce job [11]:

Step 1:

Login to the dedicated hadoop system user (hduser) , start Hadoop.

```
su - hduser
start-all.sh / start-dfs.sh /start-yarn.sh
```

Step 2:

Create a .java file in your local directory using nano

```
nano Node.java
nano Map.java
nano Reduce.java
nano Main.java
```

Step 3:

Create working directories in HDFS.

```
hadoop fs -mkdir /user/hduser/pagerank /user/hduser/pagerank/input
```

Step 4:

Write the input files

```
hadoop fs -put ady/Documents/input.txt /user/hduser/pagerank/input
```

Step 5:

Create a temporary build directory locally

```
mkdir pagerank_classes
```

Step 6:

Compile the target Java program

```
javac -cp $HADOOP_CLASSPATH -d pagerank_classes
                                /home/hduser/com/pagerank/*.java
```

Step 7:

Create a JAR archive containing the Mapper, Reducer, and driver classes

```
jar -cvf pagerank.jar -C pagerank_classes/ .
```

Step 8:

Execute Hadoop

```
hadoop jar pagerank.jar com.pagerank.Main /user/hduser/pagerank/input/input.txt
                                          /user/hduser/pagerank/output
```

References

[1] http://link.springer.com/article/10.1007%2FBF03254946

[2] http://ieeexplore.ieee.org/xpl/freeabs_all.jsp?arnumber=4215714&abstractAccess=no&userType=inst

[3] http://www.aifb.kit.edu/images/0/03/2005_1001_Tempich_Community_based_1.pdf

[4] http://www.academia.edu/459912/Improving_the_Performance_of_the_Peer_to_Peer_Network_by_Introducing_an_Assortment_of_Methods

[5] http://snap.stanford.edu/data/p2p-Gnutella04.html

[6] http://citeseerx.ist.psu.edu/viewdoc/summary?doi=10.1.1.147.3230

[7] http://dl.acm.org/citation.cfm?id=1772730

[8] http://dl.acm.org/citation.cfm?id=1099705

[9] http://link.springer.com/chapter/10.1007/978-3-319-06653-0_19

[10] http://www.michael-noll.com/tutorials/running-hadoop-on-ubuntu-linux-single-node-cluster/

[11] http://www.bogotobogo.com/Hadoop/BigData_hadoop_Install_on_ubuntu_single_node_cluster.php

[12] http://pr.efactory.de/e-pagerank-algorithm.shtml